

SIM: The Book

Architecture, kernels, and reusable modules

Hob Nilre & Bo C. Herlin

2026-09-30

Book draft: chapters 1-2; remaining chapters planned.

PDF created: 2026-10-01 10:55:31 UTC

Latest on GitHub: <https://github.com/hobnilre/rust-sim-the-book>

Contents

1	SIM: Make the Kernel Explicit	1
2	The SIM Kernel	14
3	Libraries and the Loader	33
4	Expressions and Codecs	34
5	Evaluation and Execution	35
6	State, Identity, and Persistence	36
7	Effects, Capabilities, and Hosts	37
8	Inspection and Tooling	38
9	Building Applications from Modules	39

SIM: Make the Kernel Explicit

Build applications around reusable domain modules

Every program has rules that turn input into behavior. Those rules may form a deliberate core, or they may be scattered through event handlers, command processing and storage helpers. This chapter calls that implicit center a kernel and asks what becomes possible when we give it an explicit boundary. No general-purpose language is required: copy, paste, move and export can be ordinary operations reached from a graphical interface, a command-line tool or another program. Separating shared runtime contracts from domain behavior then lets the domain become a module that more than one application can use. Through an illustrative document editor, we explain the appeal of this design, the agreements that make reuse real, and a practical way to begin without building a speculative framework. SIM gives the idea a concrete expression in Rust, while leaving surface syntax and domain behavior open.

1.1 Your program already has a kernel

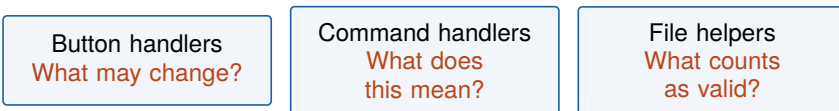
A diagram editor receives a mouse movement and changes a document. A batch tool reads a file and writes a transformed one. A service accepts a request and updates some state. Their interfaces differ, but each must decide what its inputs mean, which work to perform, what may change, and how the outcome becomes visible. Those decisions are the program's core.

In this chapter, that core is what we mean by a *kernel*. It need not be general, reusable or located in a directory with that name. A program can have its kernel spread across button callbacks, command handlers, validation routines and file helpers. The responsibilities exist even when the boundary does not.

This is the useful sense in which every program has a kernel. It is an architectural way of looking at the program, rather than a claim that every executable contains an operating-system kernel or an interpreter. At the smallest scale, the core may be no more than a few functions. At a larger scale, it may coordinate many subsystems. Naming it does not require centralizing all execution in one object or thread.

The basic picture is input, kernel, output. Input includes more than text: gestures, messages, startup configuration, a timer firing, or the state left by an earlier action can all matter. Output includes values, visible changes and effects such as writing a file. An interactive program repeats this conversation while retaining state. Figure 1.1 shows both the scattered implementation and the responsibility we can identify inside it.

The core can be scattered



The responsibility is still present

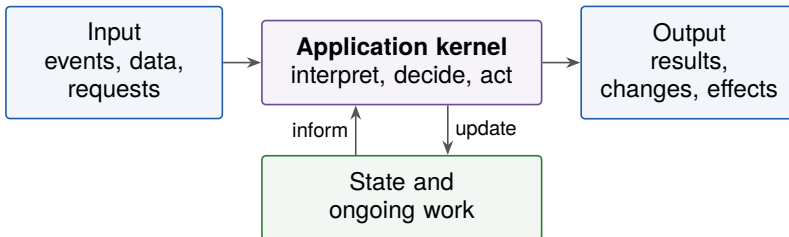


Figure 1.1. Schematic views of the same application. Core decisions can be scattered through several files; the input–kernel–output view identifies their common responsibility. State carries consequences from one interaction to the next.

The arrow picture does not make a stateful application a pure function. A request can suspend, fail, emit several events, or depend on external state. Some programs have no ongoing external input at all. The point is to identify the rules governing behavior, including initial conditions and ongoing work, so that their relationship to the outside world can be designed.

Once that center is visible, a practical question follows: if the program needs these decisions anyway, why leave their organization accidental? Why not give the core a clear boundary from the beginning, so the first interface does not become the only way the program can work?

1.2 A kernel does not need a language

It is easy to hear “runtime” or “kernel” and imagine a parser, a prompt and a programming language. None is a prerequisite. A program can expose a small vocabulary of actions through ordinary function calls. For a diagram editor, that vocabulary might include copying selected objects, pasting a fragment, moving an object, checking a document and exporting a result.

These operations already have meaning whether or not anyone can type them. They need inputs, outcomes and rules. They do not need variables, loops, macros or a Lisp reader. A textual language is one possible way of arranging calls to such operations. It can arrive later, or never.

1.2.1 Copy, paste and drag are enough to begin

Consider an illustrative editor whose documents contain shapes and connections. This editor is a design example, not a claim about a shipped SIM application. A menu item and a keyboard shortcut can both request a copy. A paste request supplies a document fragment and a destination. A drag gesture can eventually request that an object move to a new position.

The GUI knows about pointer coordinates, focus, selection gestures and clipboard formats. The document behavior knows what a shape is, which connections remain valid when it moves, and how a pasted fragment acquires fresh identities. The interface between them says what is being requested without requiring a mouse event or a window object as its argument.

Not every pointer movement has to become a committed edit. A GUI can show a temporary preview and commit one move when the user releases the pointer. Another product may need live document updates throughout the gesture. The boundary should express the

chosen interaction semantics; it should not erase them. When snapping or geometric constraints affect a valid document position, the reusable domain behavior should own those rules even if the GUI displays their effect during a preview.

Figure 1.2 shows the resulting freedom. The origin of a request changes, while its domain meaning can remain the same.

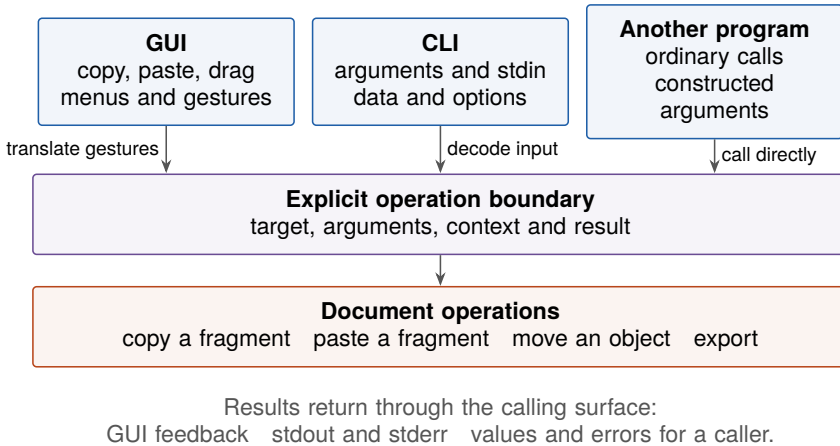


Figure 1.2. Schematic alternative surfaces for the illustrative editor. A gesture, a command-line request or a direct application call can reach the same domain operations. Only a surface that accepts a language needs that language’s parser.

This separation has a well-established architectural precedent. Cockburn’s ports-and-adapters account places an application’s meaningful interactions behind interfaces that can be driven by people, tests, batch jobs or other programs. Device-specific adapters translate those interactions at the boundary (Cockburn, 2005). Here, we use that boundary as the starting point for a further separation: shared runtime agreements and replaceable domain modules.

1.2.2 Standard streams are another surface

A command-line application can take options and arguments, read data from standard input, write its result to standard output, and report diagnostics on standard error. Rust exposes these standard

streams through its I/O library ([Rust project, n.d.](#)). They carry input and output; they do not prescribe the application's domain model or require a programming language.

For the editor example, a batch tool might read a document from `stdin`, apply a requested move or validation operation, and write the resulting document to `stdout`. Its adapter decodes bytes and command options into explicit requests. A diagnostic about an invalid move becomes `stderr` text and an appropriate exit status. The same domain failure could become a highlighted object in the GUI. The failure's meaning belongs to the shared behavior; its presentation belongs to the surface.

Keeping the ordinary result stream separate from diagnostics also makes it possible to connect the tool to another process without mixing an error message into document data. This is an interface choice the application must honor, not a property bestowed by calling its core a kernel.

A parser for file data or command options may still be needed. That is different from implementing a general-purpose language for the whole application. A direct caller can bypass textual syntax completely and submit already constructed arguments. The kernel boundary is an operation boundary, not necessarily a text boundary.

1.3 Give the center a boundary, then separate its responsibilities

So far, “kernel” has meant the program's meaningful core, including its domain rules. To reuse the same runtime structure across domains, we make a second distinction inside that core.

Some agreements concern participation: how a value is carried, how an operation is identified and invoked, where a call's context comes from, and how an error or result is described. Other rules concern the domain: what counts as a valid connection in a diagram, what copying a shape includes, or what exporting a document means. A *protocol kernel* holds the shared participation agreements. *Domain modules* provide the behavior that makes the application useful.

The application core has not vanished. It is now a composition of a small runtime center and the domain behavior attached to it. The kernel does not become a diagram editor simply because a diagram module uses its contracts. The same center can support another domain without acquiring that domain's rules as special cases.

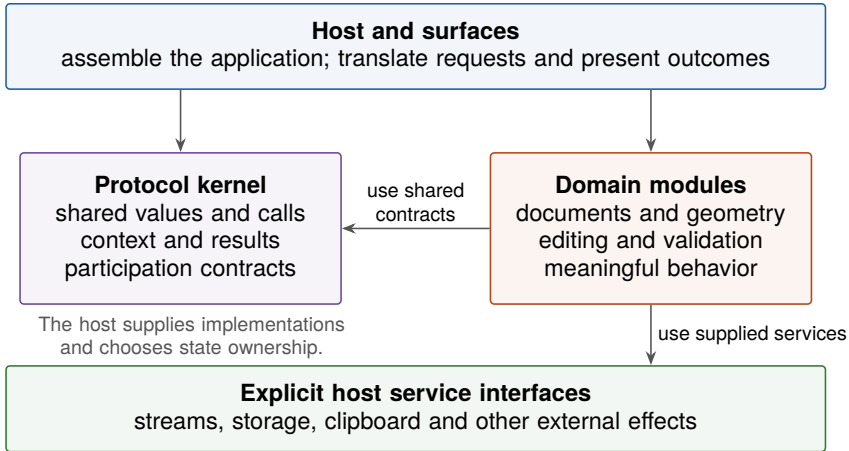


Figure 1.3. Schematic responsibility split. The host assembles surfaces and services; domain modules implement meaningful operations against shared kernel contracts. Arrows show dependencies on contracts, not a mandatory sequence through every box.

Figure 1.3 also gives the *host* a clear job. The host is the program that assembles the parts: it chooses modules, creates their context and state, connects external services and exposes an interface. A GUI host may provide a clipboard and interactive file selection. A batch host may provide explicit input and output streams. The domain module should not discover a desktop window merely to find its document.

The split is about ownership and dependencies, not just file placement. A module that reads a global “current window” to find its data still depends on the GUI, even if its source lives in a folder named domain. A module that receives a document and an explicit destination can be called by a different host without inventing a window first.

There is a useful connection here to information hiding. Parnas's decomposition criterion groups responsibilities around design decisions that should be hidden from other modules, particularly decisions likely to change. Merely turning successive flowchart steps into modules does not achieve that goal (Parnas, 1972). Our input-kernel-output drawing is therefore a way to discuss behavior, not a prescription for exactly three source modules.

In the editor, the document representation can change without changing how the GUI requests a move, provided the operation's contract is preserved. The clipboard encoding can change without changing the meaning of a document fragment. Those are concrete boundaries to design, rather than a demand that every program adopt an elaborate universal object system.

1.4 Make the domain usable away from its first interface

The payoff becomes clearer when we follow one editing operation across hosts. Suppose a document contains two connected shapes. The user copies both, pastes them into another document and moves the new pair. A reusable document module must preserve the meaning of that sequence even when no GUI is present.

1.4.1 Copy and paste need a data agreement

Copying can produce a fragment containing the selected shapes and the connection between them. It should not produce a reference to a GUI widget or assume that the destination can read the source window's memory. The fragment contract defines which properties and references travel with it.

Pasting must then decide which identities to allocate, how internal connections are remapped, and what happens to references that point outside the copied selection. One possible rule preserves internal connections and reports unresolved external references. Another may reject such a fragment. The application needs a deliberate rule, and all hosts using that version of the operation need the same meaning.

This is where reuse becomes more than sharing a function name. "Paste" on its own is not a sufficient agreement. The input, owner-

ship, result and failure behavior together define what another program can safely ask for.

1.4.2 A move needs explicit state and units

The move operation needs a target document, an object identity and a position in agreed document coordinates. A GUI converts screen coordinates through its viewport before issuing the request. A batch caller can supply document coordinates directly. The shared domain behavior checks constraints and applies the accepted change.

For a long-lived document, the contract may also need to say which revision the caller expects to change. Otherwise, another action could invalidate an object or move it between the caller's observation and its request. An explicit core does not automatically solve concurrency; it provides a place to define the chosen rule, whether that is serialization, revision checking or another policy.

The useful invariant is that each host reaches the same document rules. The hosts can still have different workflows. A GUI offers a preview and undo interaction. A batch job may stop at the first invalid request. Reusing the operation does not require reproducing the entire user experience.

1.4.3 External effects need a visible owner

Clipboard access, file writes and network requests connect the application to its environment. A reusable module can receive an explicit service, return data for the host to write, or participate in a defined effect protocol. The appropriate choice depends on the job. What matters is that the dependency is visible and can be supplied in the new host.

For example, exporting may first produce a document representation, which the GUI host saves through a selected destination and the CLI host writes to stdout. Large outputs may need streaming and cancellation rather than one in-memory result. Those needs should shape the contract before claiming the exporter works in both environments. Changing the wrapper cannot compensate for an interface that omits a necessary behavior.

Once these agreements exist, a batch tool can reuse the editor's operations without driving menu items. An automated check can

exercise document rules without generating mouse events. A second application can embed the same behavior without adopting the first application’s window layout.

1.5 Reuse becomes a property of the architecture

Imagine three products: an interactive editor, a batch document converter and a service that checks submitted diagrams. Each chooses a different surface and deployment model. All three can use the same document module where their required semantics agree. The converter can also reuse a format module; the service can reuse a validation module. Shared contracts give those modules a common way to participate.

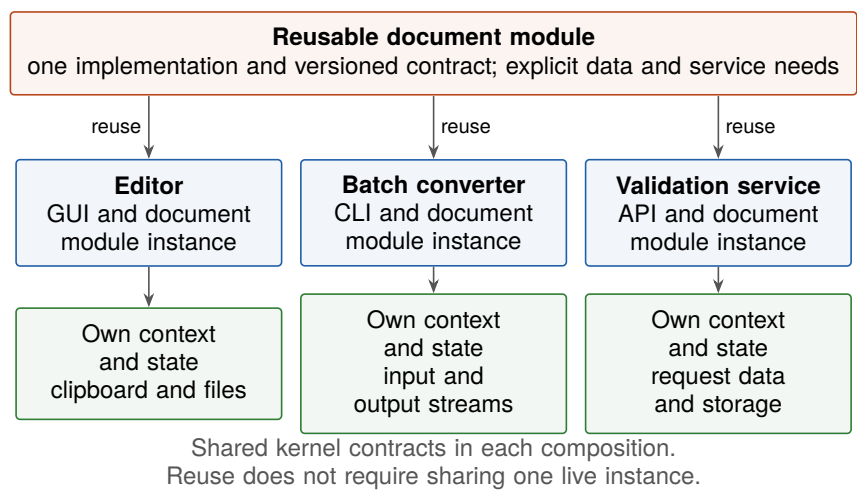


Figure 1.4. Schematic reuse across products. The same module implementation can be instantiated in different hosts, each with its own context, document state and external services. The diagram does not imply a shared process, shared mutable state or a network service.

In Figure 1.4, reuse means that the same implementation and contract can serve several compositions. It need not mean that all products share one running instance, one address space or one permissions set. The host can create separate document state and supply different services. Crossing a process or machine boundary adds

transport and serialization obligations; it is not a free consequence of the module boundary.

The benefit can compound. A correction to document validation can be made in the shared module, then adopted by each product through its normal upgrade and verification process. A new interface can reuse existing operations. A new domain module can reuse the same calling and inspection conventions. The team spends less effort rediscovering how each subsystem participates and more effort on what the new subsystem actually does.

Compatibility still has several layers. Two modules may agree on how a value is passed while disagreeing about coordinate units, identity lifetime or what a successful operation guarantees. A shared runtime can connect them; it cannot invent those missing semantic agreements. Names, versions, data contracts and integration checks are part of the reusable module's product.

There is also a useful result before a second product exists: the domain can be exercised without its first interface. That gives developers a way to discuss and check what the program means independently of how it happens to look. It makes future reuse plausible through a present boundary, rather than relying on a promise that the GUI code can be extracted later.

1.6 SIM gives the idea a concrete shape

SIM is an expandable Rust runtime built around a protocol kernel. In the inspected public implementation, version 0.4.0 at revision `bbf0967be4f8`, libraries meet through common contracts for values, objects, operations, contexts and registered exports ([SIM contributors, 2026](#)). Those contracts let a new concrete object participate without requiring each caller to know its Rust type.

An operation has an identity and a description of its expected input and required capabilities. The kernel's checked dispatch path resolves an operation, requires its declared capabilities and checks its input shape before entering its implementation. A *shape* is a runtime description used to check whether an input is acceptable. The kernel supplies the common calling machinery; the library supplies the operation's domain behavior. Other metadata, including promised

result shapes, does not become a universal post-call check merely because it appears in the description.

This is a concrete version of the separation developed above. The kernel need not contain the rules for moving a diagram shape. A library could supply that behavior while using the common value and operation contracts. The editor in this chapter remains an illustration of that possibility, not an integration demonstrated by the inspected repository.

SIM also separates the forms used to describe work from the live objects that perform it. Its shared expression representation is independent of a particular textual syntax. Concrete codecs and language behavior can be supplied around the center. Lisp can be one surface, but SIM's identity is not tied to Lisp. A host with an operation key, a target object and input values can use checked dispatch without first constructing a textual program.

The implementation contains machinery as well as interfaces: registration, stores, reference evaluation policies and bootstrap objects make the contracts usable. "Small kernel" describes a responsibility boundary, not a claim that the crate contains no behavior. The important discipline is that a new domain can contribute its own behavior without making the shared center responsible for that domain's rules.

Library manifests and export records provide a vocabulary for declaring what a module contributes. The next architectural question is how the host obtains and activates that contribution, resolves its dependencies and manages its lifetime. That is the loader's subject. A modular design can begin with ordinary static composition; dynamic loading is an additional choice with additional obligations, not a prerequisite for the argument here.

Chapter 2, *The SIM Kernel*, develops the kernel's contracts in more detail. This chapter supplies the motivation: why those contracts can be useful even for a program whose visible interface is only a few buttons or a small command-line tool. Neither chapter assumes the reader wants a language implementation walkthrough.

1.7 Build the boundary from the start; let its breadth follow the work

Making the kernel properly from the start does not require predicting every future module. It requires making today's behavior callable without smuggling today's interface into its inputs. The first version can be small enough to understand in one sitting.

Start by naming the domain's meaningful actions and the state they affect. For the editor, copying a fragment and moving an object are better starting points than reproducing the GUI toolkit's event vocabulary. Define the document identity, coordinate meaning, success result and relevant failures. Choose who owns the state and who supplies external services.

Then implement one complete path through that boundary. Let the GUI invoke it, and let a small direct caller or automated check invoke it as well. If the second caller needs a hidden window, global selection or modal dialog, the boundary still contains assumptions from the first interface. Resolve those assumptions while the operation is small.

As several operations and modules appear, identify the participation rules they actually share. A common context, result convention or operation description may now justify a shared runtime contract. Domain distinctions should remain domain distinctions. A geometry rule does not belong in the kernel merely because two editor features use it; it may belong in a geometry module.

SIM offers an existing contract layer when that degree of extensibility and inspection fits the project. For a small, closed application, ordinary typed functions and explicit dependencies may already provide the needed boundary. The useful decision is whether several independently evolving modules and surfaces need shared runtime agreements. There is no measured claim here that one arrangement is always faster, smaller or cheaper to maintain.

The design also has costs: another interface to understand, contracts to keep compatible, and sometimes additional indirection or runtime checking. A universal dispatcher that merely forwards every call while hiding useful types may add little. The boundary

earns its place when it makes real behavior usable in another context and keeps changes with the component that owns their meaning.

What makes the idea appealing is that the application stops being trapped in its first presentation. The editor's knowledge of documents can outlive its first window. A command-line tool can become a useful part of another program. A new surface can offer existing behavior in a new way, and a new domain can join an already understood runtime.

The program had a core all along. Giving it a deliberate boundary turns that core into something we can name, compose, inspect and reuse. The domain remains the reason the program exists; making it a module gives that domain more places to work.

References

1. Cockburn, A. (2005). *Hexagonal Architecture*, HaT Technical Report 2005.02. [Original article](#), dated 4 September 2005. Accessed 29 September 2026.
2. Parnas, D. L. (1972). "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM*, 15(12), 1053–1058. [doi:10.1145/361598.361623](#).
3. Rust project (n.d.). *Rust standard library documentation*, module `std::io`. [Official documentation](#). Accessed 29 September 2026.
4. SIM contributors (2026). *sim-kernel*, package version 0.4.0, inspected revision `bbf0967be4f8`. [Public source and contract documentation](#). Revision dated 29 September 2026.

The SIM Kernel

How a Rust runtime grows through shared contracts

An extensible runtime needs more than a way to load another library. Its components must agree on what a value is, how an operation is requested, which arguments are acceptable, who may perform an effect, and how the installed system can be inspected. The SIM kernel brings these agreements together in a common Rust contract layer. This chapter follows an illustrative sensor extension through that layer, using diagrams and a short dispatch algorithm to explain the design without a Rust implementation walkthrough. The central benefit is that domain behavior, surface syntax, evaluation policy and inspection can evolve around shared protocols. We also examine the cost of that flexibility: runtime checks, explicit identity choices, truthful metadata and a firm distinction between saved descriptions and live executable objects. This chapter establishes the vocabulary for a later treatment of the loader and its lifecycle responsibilities.

2.1 A runtime that can keep growing

Suppose a developer adds a sensor library to a running software system. Reading the device is only part of the work. Other components need to find the sensor, understand its arguments, ask whether reading is permitted, consume its results, and explain its behavior to a person or an automated tool. A second library might calculate statistics over the readings. A third might provide a different input language. If each addition introduces its own representation and discovery rules, the integration work grows with the number of combinations.

SIM addresses that problem by giving its components a common set of contracts. The kernel names expressions, live values, operations, shapes, capabilities, library exports and inspection records. Libraries implement behavior against those contracts. The interesting result is a place where a new domain object can be called, checked and described using concepts that its callers already know. The kernel does not need to anticipate the concrete Rust type of every future sensor, number domain or user interface.

The word *kernel* here means the shared contract layer of the SIM runtime. It is not an operating-system kernel. SIM is an expandable Rust runtime with several possible codec surfaces; a Lisp-like notation is one possible surface, rather than the definition of the system. A *codec* translates between an external representation and the runtime’s shared forms. Changing that representation need not replace the underlying object protocols.

Figure 2.1 shows the architectural division. This is a dependency picture: components above the kernel depend on its agreements. It is not a claim that every request follows one fixed execution pipeline.

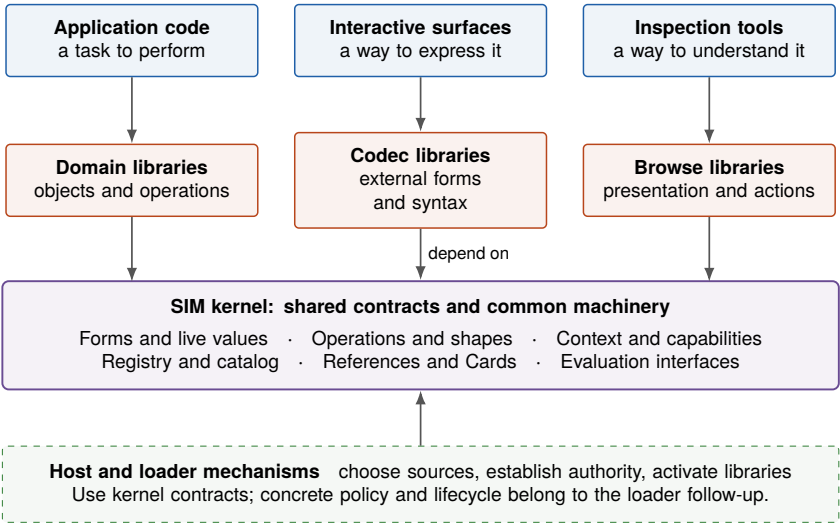


Figure 2.1. Schematic dependency view. Concrete libraries and host mechanisms meet at a common contract layer. The loader’s concrete policies are reserved for the next chapter.

This chapter explains the design at the public `sim-kernel` revision `bbf0967be4f8`, whose package version is 0.4.0. The public implementation and its tests establish the behavior described below (SIM contributors, 2026). The audience is a software developer who understands libraries and interfaces; familiarity with Rust syntax is optional.

The argument has a deliberate boundary. We start with the contracts that let an extension participate in SIM. We introduce the registration substrate and explain why saved metadata differs from live behavior. The concrete acquisition, activation and lifetime of a library belong to the loader follow-up. A future chapter can therefore begin with “how does this library enter the system?” without first rebuilding the vocabulary of values and calls.

2.1.1 What is actually in the center?

The kernel is more than a collection of empty interfaces. It includes common representations, stores, checked dispatch machinery, reference evaluation policies, and bootstrap list and table objects. Those implementations make the contracts usable before richer libraries are installed. The important boundary is responsibility: concrete language grammars, number-domain arithmetic, application behavior and host loading policy can be supplied outside the center.

That boundary is useful because the things that change for different reasons can have different owners. A sensor author defines device behavior. A codec author defines a surface representation. A host decides which capabilities it will grant. An inspection library presents structured descriptions. They still need to agree on names, values and operations, but none needs to own all the others’ implementations.

There is no measured claim here that this organization is faster, smaller in compiled bytes, or easier to maintain than every alternative. Its benefit is an architectural one: common consumers can work against stable participation rules as new concrete kinds are introduced. Its costs will become visible along the same path.

2.2 One extension, several cooperating contracts

Consider an illustrative library exposing a sensor object. The object offers a versioned operation, `sensor:sample@v1`, that accepts a positive sample count and returns a batch of readings. Reading the device requires the capability `sensor.read`. A separate operation, `signal:mean@v1`, accepts an already captured nonempty batch and calculates its arithmetic mean. The count, batch shape, capabilities and operation names are design choices for this example; they are not a claim that the kernel ships a sensor driver or these operations.

For a synthetic three-sample response (2,4,6), the second operation returns 4. Nothing about that arithmetic requires the kernel to know a sensor’s Rust structure. What matters is that the participating libraries agree on the batch representation, number domain and operation contracts. Figure 2.2 follows that agreement from a request to reusable data.

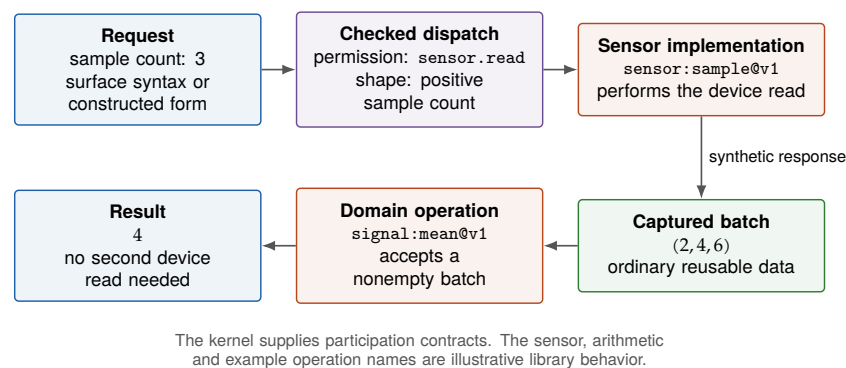


Figure 2.2. Schematic extension example. Sensor sampling is an effectful operation; reducing an already captured batch is a separate domain operation. The values are synthetic, not device measurements.

The kernel contributes five different kinds of agreement to this small story:

Table 2.1. The kernel’s contributions to one extension. Supplying the driver, arithmetic and external syntax remains library work.

Agreement	Question answered in the example
Expressions and values	How does a written request become a form and refer to a live sensor?
Operations and shapes	Which operation is requested, and does its input meet the declared contract?
Context and capabilities	May this call perform a sensor read in this context?
Evaluation policy	When is a delayed sampling request actually performed?
Registry and inspection	What was registered, and how can a tool describe it?

A surface language might express the request as a function call; an application might construct the equivalent form directly. That choice is separate from the operation key used for object dispatch. A registry symbol identifies something that can be looked up; an operation key identifies a behavior requested of a target object. A language library can connect them, but their roles should not be collapsed into one global function name.

This distinction becomes valuable when one live object supports several operations, or when two objects implement the same operation contract. The caller can ask for behavior without first discovering a concrete Rust type and writing a branch for each possible implementation.

2.3 Forms, live objects and names for things

An extensible runtime needs several representations because it asks several different questions. “What was written?”, “what object is available now?” and “how can this thing be named later?” have different answers in SIM.

2.3.1 Expressions carry form; values carry live behavior

An `Expr` is a codec-neutral expression graph. It can represent literals, collections, calls, operators, blocks, quotation, annotations and

tagged extension forms. Source locations are carried by separate located wrappers. A number literal includes a domain symbol and a textual representation; that representation does not itself implement arithmetic. A concrete number domain supplies the relevant interpretation and behavior.

This gives codecs a common target. Two suitable codecs can express the same form using different external syntax. It does not follow that arbitrary languages have identical semantics, or that every decoder accepts every form. Those are obligations of the particular codecs and evaluation policies. The kernel supplies the meeting point at which those obligations can be stated.

A `Value`, by contrast, is a reference-counted handle to a live runtime object. The object's concrete type can come from a library. Its shared surface includes identity information, operations, claims, an optional data snapshot and a human-readable display. Existing typed protocol views are also available through compatibility adapters. This is an open object model for behavior, while the expression graph deliberately has a known vocabulary plus an extension form.

Only a little Rust is needed to understand the mechanism. Rust trait objects allow different concrete implementations to be reached through a shared interface ([Rust project, n.d.-a](#)). SIM stores a shared handle to such an object. Reference counting extends its lifetime while handles remain; it does not make the object's internal state immutable. `Arc`'s ownership properties must be distinguished from the thread-safety of the data it holds ([Rust project, n.d.-b](#)). SIM additionally requires its runtime objects to satisfy Rust's thread-sharing bounds, which still does not prove application-level ordering or consistency.

The practical advantage is that adding a new object kind need not add a new variant to a universal value enumeration and then update every consumer's case analysis. Consumers that use the shared operations can already carry and interact with the new value. Concrete downcasting remains possible, but using it as the routine integration mechanism would give up much of that advantage.

2.3.2 Equality depends on the question

Two expressions can compare canonically even if the entries of a map, or the members of a set, appear in a different order. A `Value` uses object identity for equality and hashing: two handles to the same allocation compare equal. Separately allocated objects with identical contents need not do so. Therefore “these batches contain the same samples” is a domain-level question, not an automatic consequence of comparing two runtime handles.

A `Ref` supplies explicit naming choices. A symbol can be resolved through a registry; a content identifier names immutable data; a handle identifies a live object in a host-controlled namespace; a ranked coordinate names a position in a declared space. Figure 2.3 keeps these roles separate.

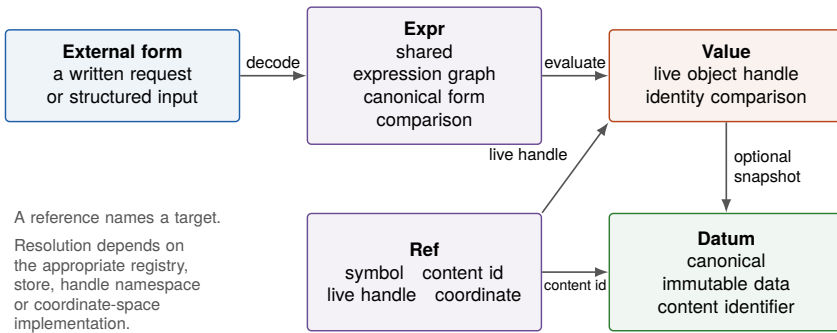


Figure 2.3. Schematic representation boundaries. Forms describe requests, values carry live behavior, and references name targets. A data snapshot is an explicit optional projection, not an automatic serialization of a live object.

For the sensor example, the registry symbol can name the sensor service, while a handle can identify the particular live connection currently in use. The captured sample batch may also have a data representation. If that data belongs to the kernel’s `Datum` vocabulary, its canonical bytes can be hashed to obtain a content identifier. A datum is a restricted data form, not an encoding of arbitrary executable expressions or Rust objects. Its canonicalization rules also reject duplicate map keys and duplicate set members.

The choice changes what can survive a restart. Content data can be stored and looked up by its identifier. A live device connection cannot be restored merely by saving the address of its object. Handles are allocated from caller-supplied seeds and sequences, so the host must manage those namespaces; a handle is not a globally resolvable address or evidence that an object is portable.

This is a strength of the design when used explicitly. SIM does not require a live object, its description and its saved state to pretend to be the same thing. A caller can choose the identity needed for the task and state which resolver or reconstruction mechanism will be required later.

2.4 A checked call has a visible boundary

An operation has a key consisting of a namespace, a name and a version. Its specification describes a subject, an input shape, a promised result shape, declared effects and required capabilities. The object's implementation supplies the behavior. The checked dispatch entry point brings that specification into the calling path.

For `sensor:sample@v1`, an input shape might accept a positive integer count; the result shape might describe a batch with a declared numeric representation. The effect declaration can identify sampling, and the capability requirement can demand permission to use the sensor. These statements give callers and tools something concrete to inspect before they know the implementation.

2.4.1 The small algorithm that matters

The order of checks is part of the contract. The following pseudocode shows the kernel's checked operation entry point, with Rust details removed:

```
invoke(context, target, key, input):
    operation = resolve target's operation for
        key
    require every capability declared by
        operation
    check input against operation's argument
        shape
    return operation's authorized
        implementation(context, input)
```

Resolution first looks for an operation directly offered by the object. If none is available, a kernel adapter may expose an existing typed protocol through the same operation interface. If neither path supplies the requested operation, resolution fails. This bridge allows existing callables, shapes and collection views to participate while callers adopt a common operation vocabulary.

For the sampling example, the absence of `sensor.read` prevents entry into the operation body. So does an input that the declared shape rejects. An unresolved shape reference is an error; an unknown shape is not silently treated as “accept anything.” Explicit universal shape references provide the intentional escape hatch. The checked path therefore distinguishes a broad contract from a broken contract reference.

That is an admission boundary for the operation body, not a blanket claim that nothing can happen earlier. Resolution and shape checking have their own implementations, and the shape protocol can describe effectful matchers. Code that calls an authorized implementation directly is also responsible for having performed the checks. The published Rust method name is not an access-control barrier against arbitrary code already running in the same process.

2.4.2 A shape can do more than return yes or no

A shape can check a live value or an expression. Its match result can carry acceptance, a score, named bindings and diagnostics. The

same family of contracts can therefore serve argument validation, grammar matching, binding and selection among alternatives. Concrete matchers and the rules for using their scores still belong to their implementations and consumers.

For a sample request, a shape could reject a missing count with a diagnostic, accept the requested count and bind it under a name, or recognize an expression before it is evaluated. The attractive part is the reusable result vocabulary. A tool does not have to learn an unrelated diagnostic convention for each application domain. Reuse of the protocol does not guarantee that two libraries chose compatible numeric units, field names or meanings; those agreements must still be designed.

This dynamic check complements Rust's static type system. Rust verifies the implementation-level interfaces. A SIM shape describes the runtime-level value or form accepted at an extensible boundary. A program can compile correctly and still submit a SIM value that the operation's shape rejects.

2.4.3 Metadata and enforcement are different

At the inspected revision, the checked operation function enforces required capabilities and the input shape. It does not perform a general post-call check of the declared result shape, and it does not automatically audit all emitted effects against the specification's effect list. Those fields remain valuable contracts for implementations, tools and other checking layers. They must not be presented as universal guarantees established by this dispatcher.

The result of an operation is also explicit. The kernel can represent a completed value, a batch of events, or a suspended step carrying an effect. Defining those outcomes gives callers a shared vocabulary; arranging event consumption and resumption still requires the surrounding runtime. A sensor operation that can suspend needs a consumer that understands that outcome. Merely having a common return type does not make every caller capable of handling every effect.

Table 2.2. What the operation specification means in the inspected implementation. Different higher-level paths may impose additional checks.

Specification field	Role at the checked operation boundary
Key	Selects behavior on the target, directly or through an adapter.
Required capabilities	Checked before entering the operation body.
Argument shape	Resolved and checked against the input.
Result shape	States the implementation's promised output contract.
Declared effects	Describes possible effects; this entry point does not audit the complete execution.

2.4.4 The host retains the power to grant

Capabilities are carried in an evaluation context. In a normal build, the host obtains a separate grant seat when it constructs a context, and the seat is bound to that context. A callable receives the context, not that seat. Creating a different context and its seat does not permit granting capabilities into the original one. Test-only support deliberately exposes fixture shortcuts and should not be confused with this production contract.

This makes authority assignment a visible host responsibility. A browser may be allowed to describe a sensor without being allowed to read it. A sampling operation can ask for its specific capability rather than assuming that any caller able to find the object may use the device. Capability sets can also be narrowed by intersection; narrowing does not manufacture a missing grant.

These are protocol-level controls inside SIM. They are not an operating-system sandbox around an arbitrary native library. Host APIs, direct in-process calls, transport isolation and the loading of trusted or untrusted code have their own boundaries. That distinction will matter particularly in the loader chapter.

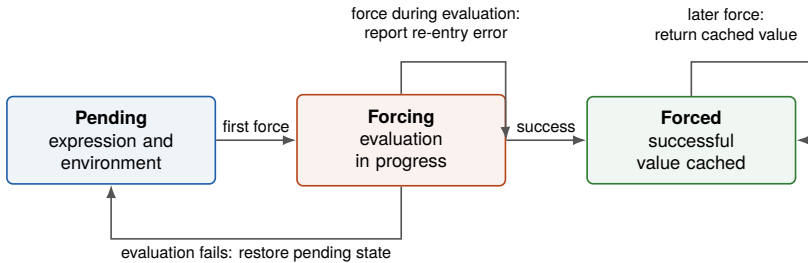
2.5 Choosing when work happens

The sensor example also explains why evaluation strategy belongs in the architecture. A delayed expression that samples a device is

observably different from an already captured batch. If the expression is forced twice, should it sample twice, or should it reuse the first successful result?

The kernel makes evaluation policy explicit. A policy prepares arguments, forces values according to demands, evaluates expressions, and controls whether macro expansion is allowed in a particular phase. The supplied reference policies include eager evaluation, non-memoizing laziness, memoized call-by-need, and strategies that mix evaluated and deferred or quoted arguments.

A *thunk* is a delayed computation that carries an expression and its captured lexical environment. Under the non-memoizing lazy policy, forcing it again runs the captured expression again. Under the supplied call-by-need policy, the first successful result is cached. Thus, for an illustrative expression that performs one sampling operation per evaluation, two forces can mean two device reads under the former policy and one successful read reused under the latter. That is a semantic choice, not merely a performance optimization.



Failure can be retried. Earlier external effects are not undone by restoring the pending thunk.

Figure 2.4. State transitions of the supplied memoizing thunk. A successful force caches a value; a failed force restores the pending computation so a later request can retry. Re-entering a computation while it is being forced reports an error.

Figure 2.4 shows the precise limit of “evaluate once.” The implementation caches a successful result. A failed attempt returns the thunk to its pending state, allowing a retry. An effect performed before a failure is not thereby undone. Memoization is therefore not an exactly-once guarantee for an external device operation. The forc-

ing state also detects re-entry instead of silently recursing; it is not a general promise that competing callers wait for a result.

Demands state what a caller needs from an argument: leave it untouched, retain expression form, obtain a value, obtain truth, or require a specified class or shape. The reference policies interpret those requests differently during argument preparation. A demand vocabulary is an interface for evaluation choices, not proof that every policy enforces every requested property at every stage. The explicit forcing path performs the corresponding reference checks.

For the sample-and-mean example, this suggests a practical design rule: capture the batch once when the application intends to analyze one observation, then pass that value to the statistics operation. Defer the sampling expression only when repeated evaluation, memoization and retry behavior are part of the intended semantics. The representation chosen by the caller should make that choice visible.

The same separation helps symbolic work. A reference policy may keep an unbound symbol, or an application of an unbound operator, as data. A stricter policy can reject it. This allows a host to distinguish a symbolic construction from a programming error without identifying one surface syntax with one immutable language policy. It does not make every expression meaningful under every policy.

The kernel also defines an evaluation-fabric interface and a realization request vocabulary for callers that should not depend on a specific transport. These contracts carry such concerns as observation mode, limits and required capabilities. They provide a place for local or remote implementations to meet; they do not, by themselves, implement a distributed scheduler or make a process-local handle remotely usable. Detailed execution and transport behavior belongs in later chapters.

2.6 A system that can describe what is present

A collection of extensible objects is much easier to work with when the system can describe its installed state. In SIM, this is not confined to debug strings. The kernel supplies both a registry substrate and

structured descriptions of runtime subjects. They answer different questions and should remain distinct.

2.6.1 Registration has an authoritative record

The registry presents typed lookup and registration interfaces, but its authoritative storage is a catalog of named tables and rows. Row data is represented through expressions; private live cells can hold objects that cannot be serialized as ordinary data. The registry maintains projection caches where its borrowed Rust interfaces need stable map or slice views. Those views serve the API; they are not separate authorities for what was registered.

For the sensor extension, registration records what the library declares and which exports were resolved to live values. Export states can distinguish resolved, declared, unsupported and invalid entries. This is more informative than a single boolean “loaded” flag: a tool can report the difference between an advertised interface and behavior that is actually available.

The catalog has explicit write policies. Mutable tables permit replacement and deletion. Sealed and append-only policies restrict changes to existing entries; derived tables reject direct writes. A catalog transaction first validates its batch, then applies it at one new epoch and records its operations in a journal. The implementation tests exercise failed batches leaving no partial rows, sequence changes or journal entries. This is an in-memory transaction boundary; it is not a claim of disk durability or a distributed consensus protocol.

Library registration adds a staged load transaction and a linker through which an implementation declares exports and supplies live values. The receiving registry changes when the transaction commits. Discarding an uncommitted staged registration leaves that registry unchanged. These guarantees concern the registry state: an external side effect performed by library initialization is not automatically rolled back with it.

That is enough loader context for this chapter. A library has a declaration, a staged way to contribute behavior, and an authoritative place where the committed result can be found. Choosing the

source, crossing a native or guest boundary, and managing activation and lifetime are the next layer's questions.

2.6.2 A snapshot describes; it does not resurrect

A catalog snapshot contains table specifications, row data, sequences and an epoch in deterministic expression form. Live payloads become explicit unresolved markers rather than being serialized as Rust objects. Restoring such a snapshot recreates data rows. It does not reopen the sensor connection or restore an executable closure from its display text.

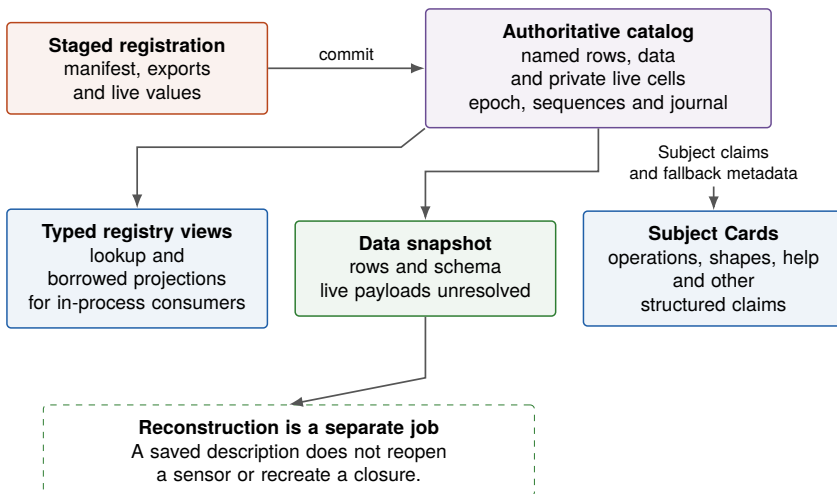


Figure 2.5. Schematic registry views. Typed lookups, data snapshots and subject Cards serve different readers. Saving a catalog description does not save live executable payloads; reconstruction requires a separate loader or host mechanism.

The distinction in Figure 2.5 is particularly useful for honest inspection. A saved record can say which export was expected, where it belonged and that its live payload remains unresolved. It need not pretend the behavior has been restored. Catalog deltas can describe changes between compatible epochs, but applying data changes still does not solve arbitrary object reconstruction.

Boot receipts address a related requirement. They record library identities, requested and resolved sources, dependencies and com-

mitted exports. They provide input to replay through loaders rather than claiming to serialize live Rust behavior. Their existence helps explain why the follow-up should focus on the loader: saved descriptions become useful operationally only when the system can resolve, validate and reconstruct the intended participants.

2.6.3 Cards give tools ordinary data to inspect

A Card describes a subject through ordered fields such as kind, help, argument and result shapes, tests, operations, required capabilities and related subjects. It is an ordinary runtime value, not a specially formatted paragraph that every consumer must parse. Its fixed schema provides a common starting point even when the available information is minimal. Claims and fallback data supply richer descriptions where they exist.

For the sensor object, a Card could let a tool display the sample operation, explain the count argument, show the required permission and offer a link to the batch description. A human browser and an automated assistant can consume the same structure for different presentations. A `shape-known` field helps a consumer distinguish known argument information from a generic fallback.

The kernel owns the record contract. Rich browsing, help presentation and test execution remain library behavior. Capability-gated visibility and structured redaction are part of the documented browse convention; the mere existence of a Card does not authorize exposing every field or running the listed tests. Likewise, a description is evidence of what an implementation declares, not proof that it behaves correctly.

This is the second payoff of the common contracts. The integration work done to make an operation callable also makes it describable in a vocabulary shared by tools. The runtime can carry useful information about itself as data, while leaving presentation and execution policy to their owners.

2.7 What the design buys, and what it asks of its users

The kernel's appeal is the combination of these agreements. Open objects permit new concrete kinds. Operation keys let callers request

behavior. Shapes make admission rules explicit. Contexts carry permission and evaluation choices. Catalog records and Cards make participation visible to tools. Each contract is useful separately, but their shared use is what reduces the need for a separate integration story around every library.

Consider adding a second source of sample batches, such as a saved recording. If it supplies the same batch contract, the statistics operation can continue to consume batches without knowing whether they came from a device or a file. The acquisition operations can keep different capabilities and effects. A new codec can offer another notation for the request without requiring the statistics author to implement that notation. An inspection tool can present both producers using their shared metadata. These are consequences of compatible contracts, not automatic compatibility between arbitrary implementations.

The design asks for care in return. Names and versions must be maintained. Operation metadata must describe the actual implementation. Shapes must have well-defined semantics, and missing shape information must be handled honestly. Applications must distinguish live identity from equal data and choose appropriate policies for delayed effects. Indirect calls, reference counting, metadata storage and runtime validation also have costs; this chapter makes no benchmark claim about their magnitude.

For a small closed program whose types and operations are known together, direct Rust interfaces and static composition may be simpler. SIM's contract layer becomes attractive when independently evolving libraries, representations and tools need to meet inside one runtime, especially when that runtime must describe what it contains. The decision is about the required kind of extension and inspection, not a general ranking of dynamic and static software.

2.7.1 What was checked for this account

The source revision was preserved and its kernel test suite was run in an isolated copy with Rust 1.96.0. The inspection traced the chapter's central claims through the object and operation contracts, shape resolution, capability grants, thunk transitions, catalog

transactions, snapshot handling and Card projection. Tests include failures that prevent operation-body execution, shape-reference errors, context-bound grants, successful thunk memoization, retry after a failed force, failed transaction atomicity and data-only snapshot restoration. These are exercised implementation behaviors, not a proof of every possible library's correctness.

The sensor and mean example is illustrative. The three sample values are synthetic. No sensor hardware, concrete device library, benchmark or distributed deployment was tested for this chapter. The distinction matters because the kernel's contribution is the participation contract; validating a particular device or transport requires the implementation that supplies it.

2.7.2 The next question belongs to the loader

We can now state the loader's job precisely. A prospective library must become a set of registered, usable participants in these contracts. The system needs to choose and identify its source, account for dependencies and versions, cross the relevant execution boundary, commit the available exports, and handle failures and lifetime changes. Saved boot descriptions must reconnect to actual behavior rather than masquerading as it.

Those questions deserve their own chapter. This chapter establishes why they have a coherent place to land. The SIM kernel gives an expanding runtime a shared language for *what a component can be asked to do, under which conditions, and how that participation can be inspected*. Its value is that new behavior can join a system that already knows how to ask those questions.

References

1. SIM contributors (2026). *sim-kernel*, package version 0.4.0, inspected revision `bbf0967be4f8`, 29 September 2026. [Public source and contract documentation](#). The implementation and its tests are the primary source for the architecture described here.
2. Rust project (n.d.-a). *The Rust Programming Language*, “Using Trait Objects to Abstract over Shared Behavior.” [Official documentation](#). Accessed 29 September 2026.
3. Rust project (n.d.-b). *Rust standard library documentation*, `std::sync::Arc`. [Official documentation](#). Accessed 29 September 2026.

Libraries and the Loader

Expressions and Codecs

Evaluation and Execution

State, Identity, and Persistence

Effects, Capabilities, and Hosts

Inspection and Tooling

Building Applications from Modules